

服务器 IP 地址：10.102.125.185

服务器程序：图 5.12, 5.3, 5.11；客户程序：图 5.4, 5.5

实验二 TCP 回射客户-服务器程序设计

(实验日期：2026 年 3 月 25 日)

一、实验目的

1. 掌握 TCP 回射客户-服务器程序的基本结构，掌握并发服务器程序设计的基本方法；
2. 理解信号的概念，理解 wait 和 waitpid 函数的区别；
3. 掌握数值型数据的网络传输方法。

二、实验内容

1. 编写 TCP 回射客户-服务器程序；
2. 编写 SIGCHLD 信号处理程序；
3. 编写整数相除子函数。

三、实验步骤

1. 编写 TCP 回射服务器程序

1) 打开会话窗口，登录远程 Linux 服务器(服务器程序容器)，**进入 workspace 文件夹** (cd workspace)，在 workspace 下建立一子文件夹，若子文件夹的名称是 tcpechoserv，可使用命令 mkdir tcpechoserv 创建。

2) 进入 tcpechoserv 文件夹 (cd tcpechoserv)，使用 vi 编辑器编写 TCP 回射服务器主函数 (vi tcpechoserv.c)、回射子函数 (vi str_echo.c) 和 SIGCHLD 信号处理子函数 (vi sigchldwaitpid.c)。

3) TCP 回射服务器程序编写完成后，进行编译 (gcc tcpechoserv.c str_echo.c sigchldwaitpid.c -o tcpechoserv -lunp)。

4) 编译结果显示程序有错误时，结合错误提示，分析错误原因，修改程序。编译成功后，运行服务器程序 (./tcpechoserv，**建议后台运行服务器程序，相应命令是 ./tcpechoserv &**)。

2. 编写 TCP 回射客户程序

1) 打开另一个会话窗口，登录远程 Linux 服务器 (客户程序容器)，**进入 workspace 文件夹** (cd workspace)，在 workspace 下建立一子文件夹，若子文件夹的名称是 tcpechocli，可使用命令 mkdir tcpechocli 创建。

2) 进入 `tcpechocli` 文件夹 (`cd tcpechocli`), 使用 `vi` 编辑器编写客户主函数 (`vi tcpechocli.c`) 和子函数 (`vi str_cli.c`)。

3) 客户程序编写完成后, 进行编译 (`gcc tcpechocli.c str_cli.c -o tcpechocli -lunp`)。

4) 编译结果显示程序有错误时, 结合错误提示, 分析错误原因, 修改程序。编译成功后, 运行客户程序 (`./tcpechocli 172.20.xxx.xxx`, **172.20.xxx.xxx 是服务器程序容器的 IP 地址**)。

3. 编写整数相除子函数

完成回射客户-服务器程序后, 对服务器回射子函数进行修改, 完成以下功能:

接收客户发来的两个整数, 进行相除运算 (要对除数进行判断, 除数不能为 0), 然后将结果发给客户。

可参阅课本 P116, 图 5.17。

四、实验要求

1. 实验中多运行几个 (至少 3 个) 客户程序, 使用 `ps -al` 命令查看服务器进程个数及他们之间的关系;
2. 在某一客户端, 按 `Ctrl+d`, 观察服务器的输出, 分析程序, 理解客户和服务子进程终止的流程;
3. 只运行一个客户, `kill` 该客户对应的服务器子进程, 然后在客户端输入一行文本, 观察客户的输出, 分析原因。

实验报告撰写要求:

1. 程序算法用伪代码或流程图进行描述 (描述服务器主函数、服务器整数相除子函数、客户主函数和客户子函数的算法);
2. 在“实验心得”部分, 谈谈你对 TCP 并发服务器的理解 (可从子进程创建、父子进程的分工和终止子进程的处理等三个角度进行叙述);
3. 程序运行结果截图 3~4 个即可, 浅色背景 (可以在 Xshell 里设置)。

```

#include    "unp.h"

int
main(int argc, char **argv)
{
    int                listenfd, connfd;
    pid_t              childpid;
    socklen_t          cliilen;
    struct sockaddr_in  cliaddr, servaddr;
    void                sig_chld(int);

    listenfd = Socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family      = AF_INET;
    servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
    servaddr.sin_port        = htons(SERV_PORT);

    Bind(listenfd, (SA *) &servaddr, sizeof(servaddr));

    Listen(listenfd, LISTENQ);

    Signal(SIGCHLD, sig_chld); /* must call waitpid() */

    for ( ; ; ) {
        cliilen = sizeof(cliaddr);
        if ( (connfd = accept(listenfd, (SA *) &cliaddr, &cliilen)) < 0 ) {
            if (errno == EINTR)
                continue; /* back to for() */
            else
                err_sys("accept error");
        }

        if ( (childpid = Fork()) == 0 ) { /* child process */
            Close(listenfd); /* close listening socket */
            str_echo(connfd); /* process the request */
            exit(0);
        }
        Close(connfd); /* parent closes connected socket */
    }
}

```

图 5.12 处理 accept 返回 EINTR 错误的 TCP 服务器程序最终（正确）版本

```

#include    "unp.h"

void
str_echo(int sockfd)
{
    ssize_t    n;
    char        line[MAXLINE];

    for ( ; ; ) {
        if ( (n = Readline(sockfd, line, MAXLINE)) == 0)
            return;        /* connection closed by other end */

        Writen(sockfd, line, n);
    }
}

```

图 5.3 函数 str_echo: 在套接口上回射行

```

#include    "unp.h"

void
sig_chld(int signo)
{
    pid_t    pid;
    int        stat;

    while ( (pid = waitpid(-1, &stat, WNOHANG)) > 0)
        printf("child %d terminated\n", pid);
    return;
}

```

图 5.11 调用函数 waitpid 的 sig_chld 函数最终（正确）版本

```

#include    "unp.h"

int
main(int argc, char **argv)
{
    int          sockfd;
    struct sockaddr_in servaddr;

    if (argc != 2)
        err_quit("usage: tcpcli <IPaddress>");

    sockfd = Socket(AF_INET, SOCK_STREAM, 0);

    bzero(&servaddr, sizeof(servaddr));
    servaddr.sin_family = AF_INET;
    servaddr.sin_port = htons(SERV_PORT);
    Inet_pton(AF_INET, argv[1], &servaddr.sin_addr);

    Connect(sockfd, (SA *) &servaddr, sizeof(servaddr));

    str_cli(stdin, sockfd);    /* do it all */

    exit(0);
}

```

图 5.4 TCP 回射客户程序

```

#include    "unp.h"

void
str_cli(FILE *fp, int sockfd)
{
    char        sendline[MAXLINE], recvline[MAXLINE];

    while (Fgets(sendline, MAXLINE, fp) != NULL) {

        Writen(sockfd, sendline, strlen(sendline));

        if (Readline(sockfd, recvline, MAXLINE) == 0)
            err_quit("str_cli: server terminated prematurely");

        Fputs(recvline, stdout);

    }
}

```

图 5.5 函数 str_cli: 客户处理循环

```

#include    "unp.h"

void
str_echo(int sockfd)
{
    long        arg1, arg2;
    ssize_t     n;
    char        line[MAXLINE];

    for ( ; ; ) {
        if ( (n = Readline(sockfd, line, MAXLINE)) == 0)
            return;      /* connection closed by other end */

        if (sscanf(line, "%ld%ld", &arg1, &arg2) == 2)
            snprintf(line, sizeof(line), "%ld\n", arg1 + arg2);
        else
            snprintf(line, sizeof(line), "input error\n");

        n = strlen(line);
        Writen(sockfd, line, n);
    }
}

```

图 5.17 对两个数求和的函数 str_echo